

API TESTING & SECURITY GUIDE

Complete Testing, Security, and Deployment Documentation

Table of Contents

1. [API Testing with Postman](#)
 2. [Security Hardening Checklist](#)
 3. [Deployment Guides](#)
 4. [Monitoring & Logging](#)
 5. [Performance Optimization](#)
-

API Testing

Postman Collection Setup

Import Environment Variables

```
{
  "id": "sams-environment",
  "name": "SAMS Development",
  "values": [
    {
      "key": "base_url",
      "value": "http://localhost:8000",
      "enabled": true
    },
    {
      "key": "api_base_url",
      "value": "http://localhost:8000/api",
      "enabled": true
    },
    {
      "key": "session_id",
      "value": "",

```

```
    "enabled": true
  },
  {
    "key": "user_id",
    "value": "",
    "enabled": true
  },
  {
    "key": "auth_token",
    "value": "",
    "enabled": true
  }
]
}
```

Test Collection - Authentication

Login Test with Session Capture:

```
// Pre-request Script
// None for login request

// POST http://{{api_base_url}}/login
// Body (raw JSON):
{
  "email": "teacher@sams.edu",
  "password": "SecurePass123!",
  "device_token": "fcm_token_xyz"
}

// Tests Script
pm.test("Login returns 200", function() {
  pm.response.to.have.status(200);
});

pm.test("Response has session_id", function() {
  var jsonData = pm.response.json();
  pm.expect(jsonData.data).to.have.property('session_id');

  // Save session_id to environment
  pm.environment.set("session_id", jsonData.data.session_id);
});

pm.test("Response has user data", function() {
  var jsonData = pm.response.json();
  pm.expect(jsonData.data.user).to.have.property('id');
  pm.expect(jsonData.data.user).to.have.property('role');

  pm.environment.set("user_id", jsonData.data.user.id);
});
```

```
pm.test("Response includes permissions", function() {  
    var jsonData = pm.response.json();  
    pm.expect(jsonData.data.permissions).to.be.an('array');  
});
```

Test Collection - Student Endpoints

Get Schedule Test:

```
// GET http://{{api_base_url}}/student/schedule?date=2024-01-15
```

```
// Headers:
```

```
// Cookie: PHPSESSID={{session_id}}
```

```
// Pre-request Script
```

```
var today = new Date().toISOString().split('T')[0];  
pm.environment.set("schedule_date", today);
```

```
// Tests
```

```
pm.test("Status code is 200", function() {  
    pm.response.to.have.status(200);  
});
```

```
pm.test("Response contains schedule array", function() {  
    var jsonData = pm.response.json();  
    pm.expect(jsonData.data).to.be.an('array');  
});
```

```
pm.test("Each schedule item has required fields", function() {  
    var jsonData = pm.response.json();  
    jsonData.data.forEach(function(item) {  
        pm.expect(item).to.have.property('id');  
        pm.expect(item).to.have.property('subject_name');  
        pm.expect(item).to.have.property('start_time');  
        pm.expect(item).to.have.property('end_time');  
        pm.expect(item).to.have.property('location');  
    });  
});
```

```
pm.test("Response time is less than 500ms", function() {  
    pm.expect(pm.response.responseTime).to.be.below(500);  
});
```

Automated Test Flow

```
// Collection-level pre-request script  
// Runs before ALL requests
```

```
// Check if session_id exists and is fresh  
if (pm.environment.get("session_id")) {
```

```
var existingSessions = pm.environment.get("session_timestamp") || 0;
var now = new Date().getTime();
var sessionAge = now - existingSessions;

// Refresh session if older than 1 hour
if (sessionAge > 3600000) {
  console.log("Session expired, logging in again");
  // Trigger login request
}
}
```

Security Hardening

Security Checklist

Input Validation & Sanitization

```
// ✓ Validate email format
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
  Response::error('Invalid email format', 400);
}

// ✓ Validate numeric input
if (!is_numeric($studentId)) {
  Response::error('Invalid student ID', 400);
}

// ✓ Validate enum values
$validStatuses = ['present', 'absent', 'late', 'excused'];
if (!in_array($status, $validStatuses)) {
  Response::error('Invalid status', 400);
}

// ✓ Validate date format
if (!DateTime::createFromFormat('Y-m-d', $date)) {
  Response::error('Invalid date format. Use YYYY-MM-DD', 400);
}

// ✓ Validate GPS coordinates
$latitude = floatval($lat);
$longitude = floatval($lon);
if ($latitude < -90 || $latitude > 90 || $longitude < -180 ||
    $longitude > 180) {
  Response::error('Invalid GPS coordinates', 400);
}
```

```
// ✓ Sanitize string input
$fullName = htmlspecialchars($input, ENT_QUOTES, 'UTF-8');
$description = filter_var($input, FILTER_SANITIZE_STRING);

// ✓ Validate array input
if (!is_array($attendanceData)) {
    Response::error('Attendance data must be an array', 400);
}
```

SQL Injection Prevention

```
// ✗ BAD - SQL Injection vulnerability
$query = "SELECT * FROM users WHERE email = '" . $_POST['email'] .
        "'";

// ✓ GOOD - Prepared statement
$query = "SELECT * FROM users WHERE email = ?";
$stmt = $db->prepare($query);
$stmt->execute([$_POST['email']]);

// ✓ GOOD - Named parameters
$query = "SELECT * FROM users WHERE email = :email";
$stmt = $db->prepare($query);
$stmt->bindParam(':email', $_POST['email']);
$stmt->execute();
```

Password Security

```
// ✓ Hash with bcrypt (cost factor 12)
$hashedPassword = password_hash($password, PASSWORD_BCRYPT, ['cost' =>
    12]);

// ✓ Verify password
if (!password_verify($inputPassword, $hashedPassword)) {
    Response::error('Invalid password', 401);
}

// Password Requirements
class LoginRequest {
    /**
     * Password requirements:
     * - Minimum 8 characters
     * - At least 1 uppercase letter
     * - At least 1 number
     * - At least 1 special character (!@#%&* )
     */
    public function validatePassword($password) {
        $pattern = '/^(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#%&*])(?=.
        {8,})/';
        return preg_match($pattern, $password) === 1;
    }
}
```

```

    }
}

```

Authentication & Authorization

```

// ✓ Verify session before each API call
public function verifySession() {
    if (session_status() === PHP_SESSION_NONE) {
        session_start();
    }

    if (!isset($_SESSION['user_id'])) {
        Response::unauthorized('Login required');
    }

    // Verify session exists in database
    $query = "SELECT * FROM sessions
              WHERE user_id = :user_id
              AND session_id = :session_id
              AND expires_at > NOW()";

    $stmt = $db->prepare($query);
    $stmt->execute([
        ':user_id' => $_SESSION['user_id'],
        ':session_id' => $_SESSION['session_id']
    ]);

    if (!$stmt->fetch()) {
        Response::unauthorized('Session expired');
    }
}

// ✓ Check user role/permissions
public function requireRole($requiredRole) {
    $user = Auth::user();

    if (!$user || $user['role'] !== $requiredRole) {
        Response::error('Access denied', 403);
    }

    return $user;
}

// ✓ Field-level access control
if ($user['role'] === 'student') {
    // Students can only view their own data
    if ($user['id'] !== $requestedUserId) {
        Response::error('Cannot access other student data', 403);
    }
}
}

```

CORS & Cross-Site Attacks

```
// ✓ Enable CORS only for trusted domains
header('Access-Control-Allow-Origin: https://sams-frontend.azurewebsites.net');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');
header('Access-Control-Allow-Credentials: true');
header('Access-Control-Max-Age: 3600');

// ✓ CSRF Protection with tokens
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));

// Verify CSRF token on POST requests
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
        Response::error('CSRF token invalid', 403);
    }
}
```

Rate Limiting

```
// File: includes/middleware/RateLimit.php
class RateLimit {
    private $maxAttempts = 60;    // 60 requests
    private $timeWindow = 3600;  // per hour

    public function check($userId) {
        $key = "rate_limit:$userId";
        $current = apcu_fetch($key) ?: 0;

        if ($current >= $this->maxAttempts) {
            Response::error('Too many requests. Try again later.', 429);
        }

        apcu_store($key, $current + 1, $this->timeWindow);
    }
}

// Usage in API endpoint
RateLimit::check(Auth::user()['id']);
```

HTTPS & SSL/TLS

```
✓ Force HTTPS on production
RewriteEngine On
RewriteCond %{HTTPS} off
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]
```

✓ Use secure headers

Strict-Transport-Security: max-age=31536000; includeSubDomains

X-Content-Type-Options: nosniff

X-Frame-Options: DENY

X-XSS-Protection: 1; mode=block

Content-Security-Policy: default-src 'self'

Encryption

```
// ✓ Encrypt sensitive data (face embeddings)
class EncryptionHelper {
    public function encrypt($data, $key) {
        $iv = openssl_random_pseudo_bytes(16);
        $encrypted = openssl_encrypt(
            $data,
            'AES-256-CBC',
            $key,
            0,
            $iv
        );
        return base64_encode($iv . $encrypted);
    }

    public function decrypt($encryptedData, $key) {
        $data = base64_decode($encryptedData);
        $iv = substr($data, 0, 16);
        $encrypted = substr($data, 16);

        return openssl_decrypt(
            $encrypted,
            'AES-256-CBC',
            $key,
            0,
            $iv
        );
    }
}
```

Deployment Guides

Azure Deployment

Step 1: Create Azure Resources

```
# Create resource group
$ az group create --name sams-rg --location centralindia

# Create App Service Plan
$ az appservice plan create \
  --name sams-plan \
  --resource-group sams-rg \
  --sku B1 --is-linux

# Create Web App
$ az webapp create \
  --resource-group sams-rg \
  --plan sams-plan \
  --name sams-backend-app \
  --runtime "PHP|7.4"
```

Step 2: Configure Database

```
# Create MySQL Server
$ az mysql flexible-server create \
  --resource-group sams-rg \
  --name sams-mysql \
  --location centralindia \
  --admin-user samsadmin \
  --admin-password 'SecurePass123!' \
  --sku-name Standard_B1s \
  --tier Burstable

# Create database
$ az mysql flexible-server db create \
  --resource-group sams-rg \
  --server-name sams-mysql \
  --database-name sams_db

# Import schema
$ mysql -h sams-mysql.mysql.database.azure.com \
  -u samsadmin@sams-mysql \
  -p sams_db < config/schema.sql
```

Step 3: Configure App Environment

```
# Set environment variables
$ az webapp config appsettings set \
  --resource-group sams-rg \
  --name sams-backend-app \
```

```
--settings \
  DB_HOST="sams-mysql.mysql.database.azure.com" \
  DB_NAME="sams_db" \
  DB_USER="samsadmin@sams-mysql" \
  DB_PASS="SecurePass123!" \
  ENCRYPTION_SECRET="your_random_secret" \
  FIREBASE_API_KEY="your_firebase_key"
```

Step 4: Deploy Code

```
# Push to Azure using Git
$ git add .
$ git commit -m "Deploy to Azure"
$ git push azure main

# Or using CLI
$ az webapp deployment source config-zip \
  --resource-group sams-rg \
  --name sams-backend-app \
  --src app.zip
```

Docker Deployment

Dockerfile

```
FROM php:7.4-fpm-alpine

# Install extensions
RUN apk add --no-cache \
  mysql-client \
  libpng-dev \
  libjpeg-turbo-dev \
  freetype-dev \
  icu-dev \
  && docker-php-ext-install \
  pdo_mysql \
  gd \
  intl

# Install Composer
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# Set working directory
WORKDIR /var/www/html

# Copy application
COPY . .

# Install dependencies
```

```
RUN composer install --no-dev --optimize-autoloader
```

```
# Set permissions
```

```
RUN chown -R www-data:www-data /var/www/html
```

```
EXPOSE 9000
```

```
CMD ["php-fpm"]
```

docker-compose.yml

```
version: '3.8'
```

```
services:
```

```
  app:
```

```
    build: .
```

```
    container_name: sams-app
```

```
    working_dir: /var/www/html
```

```
    volumes:
```

```
      - ./:/var/www/html
```

```
    ports:
```

```
      - "8000:8000"
```

```
    environment:
```

```
      - DB_HOST=mysql
```

```
      - DB_NAME=sams_db
```

```
      - DB_USER=sams_user
```

```
      - DB_PASS=sams_pass
```

```
    depends_on:
```

```
      - mysql
```

```
    networks:
```

```
      - sams-network
```

```
  mysql:
```

```
    image: mysql:8.0
```

```
    container_name: sams-mysql
```

```
    environment:
```

```
      MYSQL_DATABASE: sams_db
```

```
      MYSQL_USER: sams_user
```

```
      MYSQL_PASSWORD: sams_pass
```

```
      MYSQL_ROOT_PASSWORD: root_pass
```

```
    ports:
```

```
      - "3306:3306"
```

```
    volumes:
```

```
      - ./config/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql
```

```
    networks:
```

```
      - sams-network
```

```
  nginx:
```

```
    image: nginx:alpine
```

```
    container_name: sams-nginx
```

```
    ports:
```

```
    - "80:80"
volumes:
  - ./public:/var/www/html/public
  - ./nginx.conf:/etc/nginx/nginx.conf
depends_on:
  - app
networks:
  - sams-network

networks:
  sams-network:
    driver: bridge
```

Run Docker Compose

```
$ docker-compose up -d

# View logs
$ docker-compose logs -f app

# Execute database migration
$ docker-compose exec app php config/migrate.php

# Stop services
$ docker-compose down
```

Monitoring & Logging

Error Logging Setup

```
// File: config/logger.php
class Logger {
    private $logFile = __DIR__ . '/../logs/app.log';

    public function info($message, $context = []) {
        $this->log('INFO', $message, $context);
    }

    public function error($message, $context = []) {
        $this->log('ERROR', $message, $context);
    }

    public function warning($message, $context = []) {
        $this->log('WARNING', $message, $context);
    }
}
```

```
private function log($level, $message, $context) {
    $timestamp = date('Y-m-d H:i:s');
    $contextStr = !empty($context) ? json_encode($context) : '';

    $logEntry = "[ $timestamp] [ $level] $message $contextStr\n";

    error_log($logEntry, 3, $this->logFile);
}

// Usage
Logger::error('Login failed', [
    'email' => $email,
    'ip' => $_SERVER['REMOTE_ADDR'],
    'timestamp' => time()
]);
```

Performance Monitoring

```
// Measure API response time
$startTime = microtime(true);

// ... API logic ...

$endTime = microtime(true);
$executionTime = ($endTime - $startTime) * 1000; // in milliseconds

Logger::info("API execution time: {$executionTime}ms", [
    'endpoint' => $_SERVER['REQUEST_URI'],
    'method' => $_SERVER['REQUEST_METHOD']
]);

// Track database query time
$start = microtime(true);
$stmt->execute($params);
$queryTime = (microtime(true) - $start) * 1000;

if ($queryTime > 1000) {
    Logger::warning("Slow query detected: {$queryTime}ms", [
        'query' => $query
    ]);
}
```

Performance Optimization

Database Optimization

Indexing Strategy

```
-- Index frequently searched columns
CREATE INDEX idx_email ON users(email);
CREATE INDEX idx_user_id ON students(user_id);

-- Composite index for range queries
CREATE INDEX idx_attendance_date ON attendance(student_id, marked_at);

-- Full-text search on descriptions
CREATE FULLTEXT INDEX ft_description ON subjects(subject_name,
description);

-- Analyze table statistics
ANALYZE TABLE attendance;
```

Query Optimization Examples

```
-- ✗ BAD - N+1 query problem
SELECT * FROM students;
foreach($students as $student) {
    $attendance = DB::query("SELECT * FROM attendance WHERE student_id
        = " . $student['id']);
}

-- ✓ GOOD - Single query with JOIN
SELECT s.*, COUNT(a.id) as total_attendance
FROM students s
LEFT JOIN attendance a ON s.id = a.student_id
GROUP BY s.id;

-- ✓ GOOD - Use EXPLAIN to analyze
EXPLAIN SELECT * FROM attendance
WHERE student_id = 1 AND marked_at > '2024-01-01';
```

API Response Caching

```
// Cache student schedule (static data)
$cacheKey = "student_schedule_{$studentId}_{$date}";
$cached = apcu_fetch($cacheKey);

if ($cached !== false) {
    return Response::success($cached, 'From cache');
}

// ... fetch from database ...
```

```
// Cache for 1 hour (3600 seconds)  
apcu_store($cacheKey, $schedule, 3600);
```

Summary

Security & Deployment Checklist: - ✓ Input validation on every endpoint - ✓ SQL injection prevention with prepared statements - ✓ Strong password hashing (bcrypt cost 12) - ✓ Session validation on every request - ✓ Rate limiting for brute force protection - ✓ HTTPS/TLS enforcement - ✓ Encryption for sensitive data - ✓ CORS properly configured - ✓ Error logging and monitoring - ✓ Database indexing and query optimization